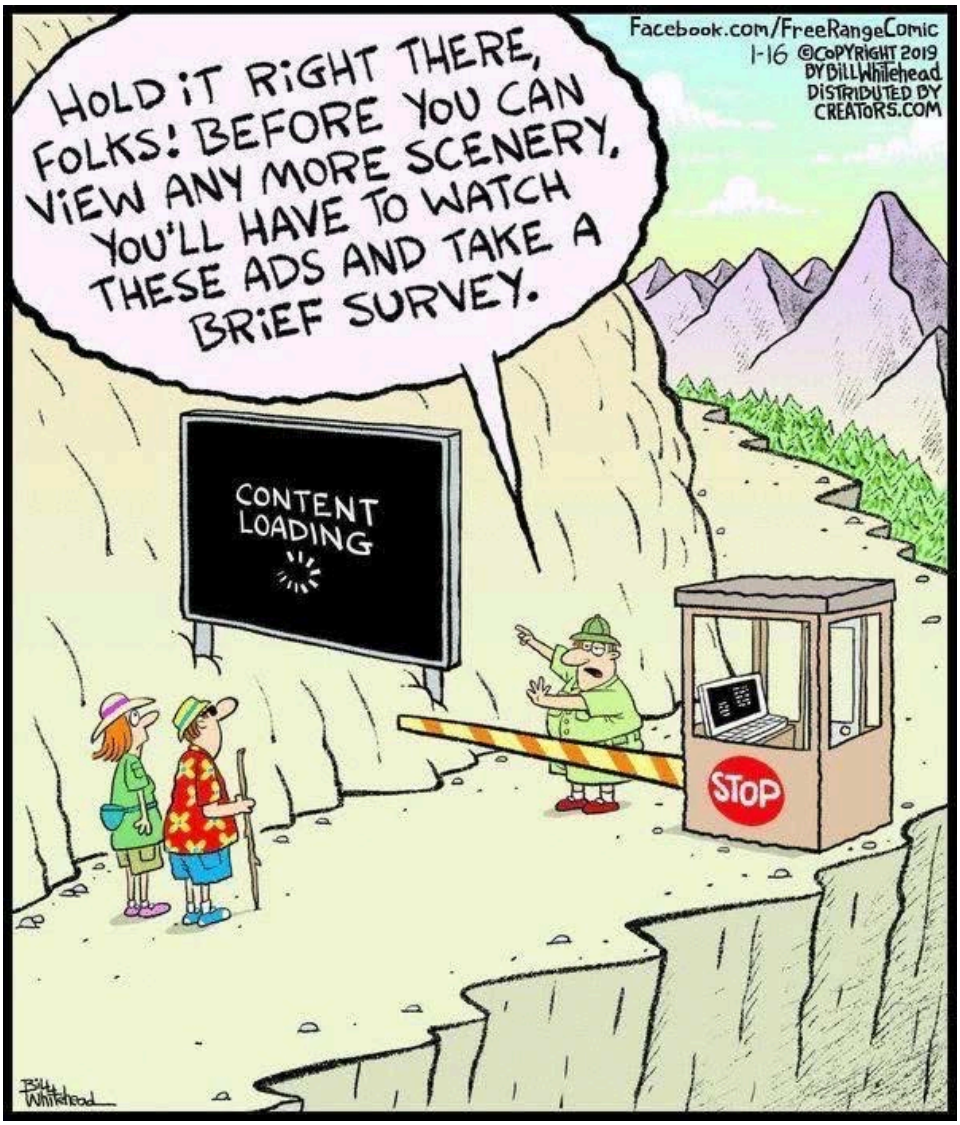
Security Now! #1060 - 01-13-26

3-Day Certificates

This week on Security Now!

• A look at Microsoft's Azure cloud code signing. • California implements DROP, global data broker opt-out. • Where's the town of "Whata Bod" Idaho. • iOS built-in Mail app worked itself out of a job. • A 30-minute tutorial for non-coders about AI coding. • Claude Code appears to be winning over the AI coding world. • Various listener musings on code signing. • A bit of Magnesium feedback. • What use are 3-day code signing certs?

It would be funnier if it didn't ring so true. Instead, it's rather sad.



Security News

Based upon the feedback I've received over the past week, we appear to have hit it out of the park with our first podcast of 2026. I received a bunch of feedback about each of the major topics we covered, and no one complained about my spending time sharing what I had learned first hand about magnesium. Many of our listeners want more. So I'll do that whenever it makes sense from time to time.

I was gratified to find a great deal of unity over what's going on in our industry regarding the shortening of certificate lifetimes and the rising costs of code signing. Since last week's three-hour podcast which couldn't have handled any more content, I stumbled upon a terrific blog post that was so "on point" that I wanted to begin this week, as I began last week, by looking at another aspect of code signing.

"Rick Strahl's Weblog" posted this past summer on July 20th, 2025 from Hood River, Oregon. Rick gave his posting the title: *"Fighting through Setting up Microsoft Trusted Signing"*. While I share what Rick wrote, please keep in mind that no matter how much this guy may sound like me and may be echoing my recently expressed sentiments, he's not me. But what he is, is further evidence that I and our many listeners are not alone and are not off base in raising an extremely skeptical eyebrow at the recent changes that have been occurring, and which will be adversely affecting everyone who wishes to author code. Rick wrote:

> *So it's that time of year (actually, the time of several years) to renew my Code Signing certificate. I always dread this because it's a manual process, and invariably, if you're not intimately familiar with the complexities of public key cryptography, the terminology is enough to drive me batty. It's gotten easier since I made some decent notes the last few times I went through this...*
>
> *But all that's out of the window this time around because the CodeSigning rules have changed.* ***Drastically!*** *It actually happened a few years ago, but I was lucky and got my local, still exportable certificate just before the rules changed so I was able to free load for nearly 3 years on the old certificate plan.*
>
> *The new rules don't allow for locally stored, exportable certificates. Instead certificates have to be served from one of a few certified online authorities, or the certs must be stored in a FIPS 140-2 Level 2+ compliant hardware security module (HSM). The keys cannot be exportable, so they effectively can't be copied and stored or used elsewhere. So you got the option of server provided keys or hardware keys.*
>
> *The idea behind this is to stop keys getting high jacked and being used by the non-originating organization. So the new keys are one time generated and non-exportable, so that they are much more restricted. Online services issue certificates that are good for only a few days when you can use them to sign with, and then automatically roll over to a new certificate.*
>
> *What all this means the complexity of getting a certificate has gotten exponentially worse, and along with that prices have gone up significantly. Base non-EV certs run in the $350-500 range with fully verified EV certificates starting around $500 per year. What used to cost me $180 for 3 years, the same provider now wants nearly $1000 for. Yikes!*
>
> ***It all seems like a huge grift.***

In this posting, Rick then quotes a separate tweet he posted to 'X' two days before his blog posting. On July 18th, Rick posted to 'X':

> *As it is, the whole CodeSigning thing has turned into another scam of Enshittification of a captured audience. If you're publishing software or even packages on NuGet now, you pretty much have to have a code signing certificate. Certificates that used to be ~$100-150 (or less for multi-year certs) per year, a few years ago, now cost $300-$400 for basic certs. The EV certs start at $500 and go up from there. The validation rules for businesses have not changed and you would think most of the expense is all in that. But this isn't about security - it's about gate keeping and just one more hurdle for a small business to have to jump over.*

So, with that introduction, Rick turns his attention to Microsoft's Azure cloud signing solution, writing:

> *Microsoft Is In the Game Too*
>
> *Microsoft who requires these CodeSigning rules in the first place for Windows SmartScreen validation and also for other things like NuGet packages, is also providing an Azure service called Trusted Signing to provide CodeSigning services, so they are on both sides of that transaction (create problem → provide a solution?) To their credit their pricing is much better than what most traditional SSL Cert providers are now charging. Azure Trusted Code Signing is still in preview but then again, it's been in preview for well over 2 years but it looks like what you see and can sign up for now is in the final stages before going to a proper release as a service.*
>
> *Microsoft Trusted Signing Certificate Pricing*
>
> *One reason to look at Microsoft's solution - despite the potential pain and suffering - is that the pricing is quite good (as of time of this post):*
>
> | *Model type* | *Basic* | *Premium* |
> |---|---|---|
> | *Base price (monthly)* | *$9.99* | *$99.99* |
> | *Quota (signatures/month)* | *5,000* | *100,000* |
> | *Price after quota is reached* | *$0.005/signature* | *$0.005/signature* |
>
> *These are non-EV, base certificates that only do basic vetting. For fully vetted EV certificates you'll need to look elsewhere. This pricing which ends up at ~$120/yr for the single cert is cheap compared to most of the SSL vendors most of which start at ~$300 for certificates with mailed hardware keys. So, you gotta give Microsoft credit here for keeping costs down and providing reasonable pricing.*
>
> *The certificates issued by Microsoft **are very short lived** - with expirations that last only 3 days to aggressively thwart invalid signing attacks in case a certificate is compromised.*
>
> *Doing a bit of research out of all the bad options out there, Microsoft's Trusted Signing seems like the least bad solution that's also cheaper than traditional certs from various SSL vendors. The good news is that it works and pricing is reasonable. The bad news: I wasted nearly an entire day trying to get it to work. Hopefully this post will help you reading this not to waste quite so much time 😄.*

Navigating the Azure Jungle

If you end up going the Azure Trusted Signing route, plan on having to wade through the Azure dependency jungle of setting up several resources and trying to understand what all the mumbo-jumbo Azure jargon amounts to. If you're doing Azure all day then much of this infrastructure dance will be familiar to you, but as someone (me!) who only occasionally jumps in for some very specific services like Trusted Signing, it's incredibly painful to deal with Azure security and the Resource dependencies and the endless nesting of services with badly defined and overlapping naming boundaries.

For Trusted Signing, finding documentation via search engines was hit or miss - the docs for this are buried behind deeply nested links - perhaps because it's still in or just out of preview (even that's hard to tell since some prompts show preview, none of the headlines do) and, also because previous releases of this technology used a completely different publishing pipeline through the Azure Key Vault.

There's official documentation although it took me a bit to discover it:

https://learn.microsoft.com/en-us/azure/trusted-signing/

This has everything you need, but the instructions require some... uhm... interpretation. The tools are terrible, and the docs don't make working with them a lot easier by making you figure out where to find files and dependencies and how to install tools.

Don't believe your lying AIs! In this day and age of AI assistants and ChatBots you would think that things like Azure configuration instructions for setting up an Azure task would be readily available. Heck there's even an Azure Specific CoPilot Model that you can use from the VS Code CoPilot integration.

But that actually yielded surprisingly bad results and did not work well with Trusted Signing either for setup or for the signing part! Part of this might be because Trusted Signing is still in Preview or because the documentation for this is almost non-discoverable and because things have changed so much with the tooling.

Long story short: After a very pissed-off day of going down many wrong paths I managed to get Trusted Signing to work for my projects, and I'll try my best to provide the details how I have this set up, hopefully sparing a few of you all the pain I ran into.

All of that that I've shared so far is just the first 10% of Rick's blog posting. Throughout the next 90% of his posting he painstakingly and very charitably details the entire process of setting up Microsoft Azure Cloud Code Signing. I have the link to his detailed instructional posting in the show notes, and I also gave it a GRC shortcut of: https://grc.sc/codesign

https://weblog.west-wind.com/posts/2025/Jul/20/Fighting-through-Setting-up-Microsoft-Trusted-Signing

Rick finally wraps up his really terrific setup walk-through with a summary that's also worth sharing. As you'll hear, some of this assumes you've managed to slog through everything that he wrote preceded it. He wrote:

The process to set up Trusted Signing was way harder than it should have been - in fact the entire process took me the better part of an entire work day. The server process is complicated primarily because the nomenclature is so crazy confusing and the dependency management on Azure is such a pain in the ass. The missing rights from the account to create an identity is particularly maddening and how you fix it even more so! But it wouldn't be Azure if you weren't cursing the thing every step of the way.

The signing process is also a pain in the ass with 3 different tool chains required. The fact that an Azure Trusted Signing command line interface add-in exists, but doesn't support actually signing is just ridiculous. With all the resources that are thrown at Azure it seems petty to not support the one feature that everybody is going to need without having to jump through hoops of managing several tool installation instructions.

But, somewhat grudgingly, I have to say that at the end of the day the process works, warts and all. Microsoft's comparatively lower pricing for the service compared to others maybe makes it worth it, and frankly the fact that I have my cert running as a service that hopefully doesn't ever need to be updated unless I quit the service is enticing. Yeah, it costs more than it did last time around - I'm now paying almost as much every year as I used to pay for three years, but given the circumstances and the Enshittification that now surrounds the entire Code Signing process, this is the best we can do for now.

I'm hoping writing this up is helpful to some, and that these instructions won't be obsolete in a few short months because Microsoft changed their designs again, as is so often the case.

Despite that I finally got it to form, one would hope they fix its performance. 5 to 8 seconds per file to sign, with no parallelism for multiple submissions, is bad. And could we please have self-contained tooling for signing? For heaven's sake, provide ONE tool that can handle the signing process in one pass without having to install 50 other things. Or better yet, have it built-in to the Azure command line interface with the trusted signing add-in that's already there.

One can hope some of this is due to the relative new-ness of Azure Trusted Signing. But... we shall see.

Rick's blog system supports reader comments , and that posting generated a bunch of feedback. I'll share the first one of the many, which read:

I'd just like to say that I've been reading your blog for probably 12 years now and I also went through this. I've learned to parse Microsoft documentation as if I were a machine and it's nice to know that someone else is turning into the cranky old man of developers.

I feel like something has been lost from the time we were excited it all worked. The days where you could slow down the genie effect on Mac with the shift key just to stare at it. When connecting to a system seemed like magic. Now we deal with artificial gatekeeping, auditing, roadblocks, deprecations for seemingly no productive reason. What happened to the joy of being excited that it all worked? Keep on trucking. But also, get off my lawn.

So, it's not just me and many of our listeners who have sensed that what's happening here is not for the benefit of the world, but for the enrichment of a few very large gatekeeping bureaucracies.

After last week's podcast I did some additional scouting around and I found that the FastSSL brand offers a standard non-EV code signing certificate for $129 per year for three years. Since it's still possible to obtain a 3-year plus 3 month certificate before March 1st, that's what I plan to do. FastSSL certificates are available from the site: https://cheapsslsecurity.com/, so that's what I plan to do. There's no longer any apparent benefit from obtaining and wielding extended validation certification, and since obtaining it means paying a lot more after first being subjected to a full body cavity search, my next code signing certificate will be the bottom of the barrel FastSSL brand. That one will take me from late February 2026 to late May of 2029. At point I'll again seek the best option and I'll certainly share my findings with everyone here.
https://cheapsslsecurity.com/fastssl/code-signing-certificate.html FastSSL: $387 ($129/yr)

DROP (Delete Request and Opt-out Platform)
https://arstechnica.com/tech-policy/2026/01/data-broker-hoarding-is-rampant-new-law-lets-consumers-fight-back/

ArsTechnica's headline was: *"The nation's strictest privacy law just took effect, to data brokers' chagrin"* with the subhead: *"Californians can now submit demands requiring 500 brokers to delete their data."* Dan Goodin explained:

Californians are getting a new, supercharged way to stop data brokers from hoarding and selling their personal information, as a recently enacted law that's among the strictest in the nation took effect at the beginning of the year.

According to the California Privacy Protection Agency, more than 500 companies actively scour all sorts of sources for scraps of information about individuals, then package and store it to sell to marketers, private investigators, and others.

The nonprofit Consumer Watchdog said in 2024 that brokers trawl automakers, tech companies, junk-food restaurants, device makers, and others for financial info, purchases, family situations, eating, exercising, travel, entertainment habits, and just about any other imaginable information belonging to millions of people.
Scrubbing your data made easy

Two years ago, California's Delete Act took effect. It required data brokers to provide residents with a means to obtain a copy of all data pertaining to them and to demand that such information be deleted. Unfortunately, Consumer Watchdog found that only 1 percent of Californians exercised these rights in the first 12 months after the law went into effect. A chief reason: Residents were required to file a separate demand with each broker. With hundreds of companies selling data, the burden was too onerous for most residents to take on.

On January 1, a new law known as DROP (Delete Request and Opt-out Platform) took effect. DROP allows California residents to register a single demand for their data to be deleted and no longer collected in the future. CalPrivacy then forwards it to all brokers.

Starting in August, brokers will have 45 days after receiving the notice to report the status of each deletion request. If any of the brokers' records match the information in the demand, all associated data—including inferences—must be deleted unless legal exemptions such as information provided during one-to-one interactions between the individual and the broker apply. To use DROP, individuals must first prove they're a California resident.

I used the DROP website and found the flow flawless and the interface intuitive. After I provided proof of residency, the site prompted me to enter personal information such as any names and email addresses I use, and specific information such as VIN (vehicle identification numbers) and advertising IDs from phones, TVs, and other devices. It required about 15 minutes to complete the form, but most of that time was spent pulling that data from disparate locations, many buried in system settings.

It initially felt counterintuitive to provide such a wealth of personal information to ensure that data is no longer tracked. As I thought about it more, I realized that all that data is already compromised as it sits in online databases, which are often easily hacked and, of course, readily available for sale. What's more, CalPrivacy promises to use the data solely for data deletion. Under the circumstances, enrolling was a no-brainer.

It's unfortunate that the law is binding only in California. As the scourge of data-broker information hoarding and hacks on their databases continues, it would not be surprising to see other states follow California's lead.

Okay. So I thought that I ought to take this out for a spin, too. Why not? As Dan wrote, and as Leo and I both discovered, it's all out there already anyway and trusting one more entity who is only asking for my information for the purpose of preventing its warehousing and resale makes sense to me. So I went over to the new drop site at: https://consumer.drop.privacy.ca.gov/.

They appear to be behind Cloudflare since I first encountered the increasingly familiar "let's verify you're human" intercept page with the spinning icon. After a few seconds it finished whatever it was doing and I was taken to the "Delete Request and Opt-out Platform (DROP)" site which identified itself with that web page title. One of the first things I noted was a "DROP Status" menu item. Clicking that out of curiosity I was taken to a short page that said: *"Enter your DROP ID to check the status of your DROP deletion request. Your ID contains 8 to 10 characters (letters and numbers)."* So that seems sort of cool. You receive a Drop ID which you can use to check back at any time. I'm thinking that I'll store that in Bitwarden's safe and also alongside my credit bureau credit freeze information.

Since this was serious business, I decided that I ought to actually read the *"Terms of Use"* fine print. And I'm glad I did, since it explains that everything I provide will be forward to data brokers and the more I provide the better job they'll be able to do of scrubbing me from their systems. This does make one take a big gulp. Here's what we're agreeing to. It says:

By using the Delete Request and Opt-out Platform (DROP), you agree to the following Terms of Use (Terms) provided by the California Privacy Protection Agency (CalPrivacy, we, us, our).

Use of DROP: By submitting a deletion request through DROP, you consent to disclosure of your personal information to data brokers for purposes of processing your deletion request pursuant to Civil Code section 1798.99.80 et seq. unless or until you cancel your deletion request. Additionally, you acknowledge that data brokers receiving your deletion request will delete any non-exempt "personal information," as defined in Civil Code section 1798.140(v), which pertains to you and was collected from third parties or from you in a non-"first party" capacity (i.e., through an interaction where you did not intend or expect to interact with the data broker).

Before submitting a deletion request, you will be required to verify you are a California "resident," as defined in section 17014 of Title 18 of the California Code of Regulations as that section read on September 1, 2017. Verification is made with assistance from state contracted third-party vendors, including Socure and Login.gov, through the California Identity Gateway.

If you are unable to confirm your California residency through these verification service providers, you may request review of your residency classification pursuant to section 7622 of Title 11 of the California Code of Regulations. You may contact CalPrivacy by visiting this webpage: https://consumer.drop.privacy.ca.gov/contactus.

In addition, you will be prompted to provide personal information, such as name, date of birth, and email address. Certain information is required to verify your residency. Otherwise, the type of information and how much you provide is up to you. However, you must only provide true and accurate information about yourself through DROP. Adding personal information about multiple people in the same request is prohibited. Information received will be used and disclosed to facilitate your request to delete and opt-out of the sale/sharing of your personal information maintained by data brokers registered with CalPrivacy. The more personal information you provide, the greater the likelihood of registered data brokers finding the personal information they maintain about you and deleting that information.

Yikes. So, if I choose to volunteer the size of my underwear, one the one hand everyone whose business it is to collect and resell such information will have that, authoritatively directly from me. The juiciest and 100% verified information directly from the source that they could ever hope to have. But because this disclosure came from CalPrivacy, its very existence means that disclose or sell it they must not, and that, in fact, they must use it solely for the purpose of identifying me and, having done so, then delete it and everything else they may have previously collected. The Terms of Use continues:

Data brokers are required to process deletion requests at least once every 45 days beginning August 1, 2026. Your submission of personal information through DROP is governed by CalPrivacy's Privacy Policy which is incorporated into these Terms by reference.

Prohibited uses — While using DROP, you agree you will not:

1. *use DROP for any fraudulent, unlawful, or prohibited purpose*
2. *impersonate any person or entity or misrepresent your affiliation with any person or entity*
3. *interfere with or disrupt the operation of DROP, or the servers or networks used to make DROP available, including threatening the integrity or security of DROP*
4. *restrict, disrupt, interfere, or inhibit any other person from using DROP*
5. *reproduce, duplicate, copy, sell, resell or otherwise exploit for any commercial purposes, any portion of, use of, or access to DROP*

Violating these terms may, on a case-by-case basis, result in restriction of your ability to access and use DROP.

Aiding another person with their request

You are only permitted to aid another consumer with their deletion request if that person has authorized you to do so and you meet the requirements described in section 7001(d) of Title 11 of the California Code of Regulations. In addition, the consumer must first have their residency verified as described in the Use of DROP section above. When aiding a consumer

with their request, you or the consumer must disclose your full name, email address, and business name if applicable through DROP when prompted. In submitting information on behalf of another person, you certify that you have authorization to do so and that the information you provide is true and correct. Adding personal information about a person who has not authorized you to submit a deletion request on their behalf is prohibited.

Third-Party Links

DROP may contain links to other websites and access to content and services of third parties, including verification services provided by our contracted vendors (Third-Party Content). We exercise no control over such Third-Party Content, and the Third-Party Content is governed by the respective third party's website terms and conditions. We are not responsible for Third-Party Content's accuracy, completeness, or legality. By using DROP, you acknowledge and agree that your use of any Third-Party Content is at your own risk. We shall not be liable for any damages arising from your reliance on or use of such Third-Party Content.

So, that's not very comforting. But that's what happens to anything when attorneys are asked to review, advise and revise. The Terms ends with something titled "*Notice at Collection of Personal Information*" which says:

YOUR DATA: When you use DROP, the California Privacy Protection Agency (CalPrivacy, we) collects personal information you enter—such as name(s), email(s), phone number(s), date of birth, zip code(s), Mobile Advertising ID(s) (MAID), Connected TV ID(s), and Vehicle Information Number(s) (VIN). We also collect usage time, device ID, and IP address.

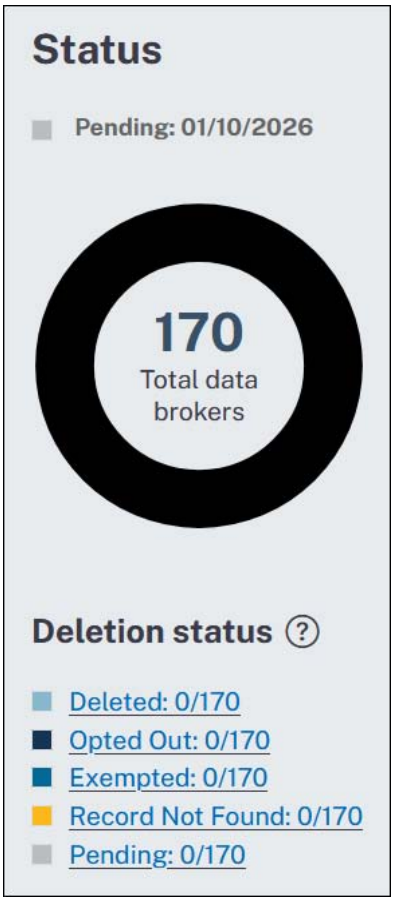
We use the data to provide your deletion request to registered data brokers, enhance the product, respond to questions, and ensure safety. Providing information and using this service is voluntary. Do not provide unrequested personal information.

YOUR RIGHTS: You may access records with your personal information. Collection is subject to the Information Practices Act and state policy. If you have any questions or concerns about this policy, contact our Chief Privacy Officer at databrokers@cppa.ca.gov.
More information at: https://privacy.ca.gov/privacy-policy/

So I did what Dan did. And it did take about 15 minutes. I chose to use Login.gov since I already have an account setup there. I don't recall why, but the email address they have for me was the one I was using in 2018. So it may have been to set up Social Security stuff in advance of my turning 65. In any event, after providing my phone number, the site used SMS to send my phone a link. Clicking that link took me to a page which requested access to my camera so that it could manage capturing the front and back of my California driver's license. It did that with a cool Arnold Schwarzenegger Terminator green grid overlay. And for each of the two captures it asked my permission to send it for verification which took a few seconds each time.

After that I was returned to the DROP page where I provided both Steve and Steven as my first names. I avoided adding "Tiberius" as my middle name, since I didn't want to confuse things. Then I provided my last name, date of birth, social security number, residence address, and Vehicle Identification Number (VIN). There were places to add a mobile advertising identification number and a SmartTV identifier. Until Apple refreshes their AppleTV hardware, which I'm holding my breath and hoping for, I'm using a ROKU. And while it ***does*** have an advertising

identifier, that number is not user displayable without side-loading a Roku channel for displaying such stuff, which was more than I was interested in doing. And I have app tracking turned off in my iPhone, so nothing to share there, either.



Once that was complete I was taken to the "Deletion request submitted" success page. I received my promised eight-character Drop ID which I can use to check back on my DROP status at any time. The page explained that beginning on the 1st of August, all registered data brokers would have 45 days to process my deletion request, that I could check my DROP status to affirmatively verify that my data had been deleted and by whom. And that I could add more information about myself at any time with any new data taking up to 90 days to process.

One cool thing is that having done this, the DROP page's menu added two new page links for "My Data Profile" and "Data Broker List" pages. The "My Data Profile" allows the provided data to be augmented at any time and it displays a ring-chart (like a pie chart but with a hollow center where we are informed that a total of 170 individual data brokers are registered with the state of California and are, thus, subject to this new law which went into effect on January 1st with an 8-month grace period. But what's most cool is that once this happens, the ring-chart has categories for Deleted, Opted Out, Exempted, Record Not Found, and Pending. I'll be very interested in seeing both the "Deleted" and the "Record Not Found" counts. They both show 0 of 170. What will they be?

The "Data Broker List" page actually displays the current status of each of these 170 individual data brokers. Though they are all currently shown as N/A, the filter option contains the same itemizations as the ring-chart. This means that it will be possible to sort and filter these 170 brokers by their declared status relative to our data.

Looking at all this, I was reminded of what Dan wrote: *"It initially felt counterintuitive to provide such a wealth of personal information to ensure that data is no longer tracked. As I thought about it more, I realized that all that data is already compromised as it sits in online databases, which are often easily hacked and, of course, readily available for sale. What's more, CalPrivacy promises to use the data solely for data deletion. Under the circumstances, enrolling was a no-brainer."*

It's somewhat creepy to be volunteering all of that information, providing it to an anti-tracking privacy enforcing facility. But that must be what any such service would need, right? After all, you cannot ask to be deleted without identifying the individual and information that you'd like to have removed. It's creepy, but I can't see any other way to go about it. And certainly CalPrivacy has the enforcement of Californians' privacy as its core mission.

AI

What about "Whata Bod"??

I wanted to share a wonderful bit of AI hallucination news from this past weekend. The National Weather Service has withdrawn a wind forecast from its social media platforms after its new AI-powered system generated a map of Idaho which included two fictitious town names "Orangeotild" and "Whata Bod".

The wind weather forecast map, which was initially shared on social media by the weather office in Missoula, Montana, on Saturday, depicted those two nonexistent towns occupying Idaho's Camas Prairie region. The forecast posting helpfully encouraged residents to "Hold onto your hats", indicating that "Orangeotild" faced a 10 percent chance of high winds, while "Whata Bod" to the south would experience calmer conditions.

Beyond the gratuitous synthesis of those two prominently featured towns, the National Weather Service's map also contained multiple spelling errors and geographical inaccuracies. The Weather Service was quick to blame these mistakes on the use of generative AI technology.

Email Client

My hMailServer crashing adventure.

Several months ago I began noticing that my beloved eM Client email client wasn't notifying me of incoming email in a timely fashion. Someone would say that they had just sent something but even after waiting a reasonable length of time nothing had arrived. I discovered that by completely closing and restarting eM Client it would then again, for a while, be reliably notifying me of newly arrived email. I haven't mentioned this until now because I hadn't been able to affirmatively verify that eM Client was the problem, though it certainly seemed to be.

Then, a few weeks ago, something caused me to look at the logs of GRC's hMailServer. What I discovered was that the server had been crashing and restarting, leaving mini-dump files behind. And before the server would crash it would log the source of its pain. It appeared to be something about IMAP and the retrieval of large file attachments. So I spent a few hours having a heart-to-heart with ChatGPT to see what it might have absorbed and chasing down the various leads it was generating. There really didn't appear to be any reason to suspect that eM Client was behind the trouble and the hMailServer discussion boards were filled with the typical threads of people commenting without actually knowing what they were talking about.

What I did realize was that if eM Client – or any email IMAP agent – was maintaining an open TCP IMAP connection, expecting to be proactively notified of newly arrived email, when the other end of the connection crashed and restarted, that TCP connection might be left hanging. eM Client would never receive the news of new email.

My environment contains both eM Client and a collection of iOS devices – iPhones and iPads – and as I was correlating the times of the server crashes with my own actions it appeared to be more connected to iOS than to my use of eM Client. Some Googling revealed that, to my surprise, iOS has historically had a surprising degree of trouble getting the IMAP protocol correct and this has been a source of great annoyance for those tending many IMAP servers.

The moment I deleted the troublesome account from all of my iOS devices, all server crashing stopped. This was about a week ago and the server has never crashed again. And my eM Client on the desktop has resumed its previous behavior of immediately notifying me of any new email.

The reason for my having dragged everyone through this sorted tale, is that my strong – in fact, overriding – proclivity is to "live off the land" wherever possible. Since every one of my iOS devices came with a built-in iOS Mail client, the last thing I would ever consider doing would be installing a second redundant email client. But, yes indeed, things had come to that. I remembered that the eM Client folks offered their mobile clients at no charge so I thought "let's give that a try."

So I downloaded eM Client from the Apple App Store. Naturally, although I specified exactly the client I wanted by name, eM Client was not first in line. No. It was preceded by sponsored apps that were paying to have my search results contaminated for their benefit. Like many others, I'm beginning to feel that the shine is fading from the Apple, which is truly sad.

Nevertheless, I was able to find, download and run eM Client. The first thing it asked upon running was whether I would like to import my existing world from a desktop instance. I thought "What?!! – Yes, please!" So it told me to open any already-configured desktop instance of eM Client, and in its menu under "Tools" I would discover "QR Export". Sure enough. My Windows desktop eM Client displayed a massive QR Code which my mobile instance saw and it was immediately set up with all of my accounts, tweaks and preferences. It's been four or five days now, since I made the switch across all five of my iOS devices and I'm still not accustomed to how much better the mobile version of eM Client is compared to Apple's built-in but uninspired mail client. eM Client even runs on a very old iPad which I have to keep plugged in, where ChatGPT's client refuses to install, scolding me that I need to update to an iOS version from this century.

GRC's email server has never again crashed, and I could not be happier. So I just wanted to give everyone a heads up that whether or not you are running eM Client on your desktop, the 100% free eM Client for iOS or Android is truly lovely. And if you're one of our many listeners who switched to the desktop eM Client after my discovery of it, or if you're one of our other listeners who wrote to rhetorically ask what took me so long? ... both groups will get the additional joy of instantaneous account and setup cloning via QR code.

Before I close this topic, I want to acknowledge that I know there is no excuse for anything some remote email client might do to cause the email server it's connected to, to crash. I am 100% aligned with that sentiment. But I love hMailServer. It's everything I want in a Windows-hosted email server. In addition to many great features, it publishes a COM interface that has allowed me to automate parts of its operation to integrate into GRC's email system solution. And because it's open source, I was able to engineer its operation to get it to do exactly what I needed. But since I'm not in the position to spend who knows how long fixing its actual problem, I'm treating its symptoms. And in this case, that worked to my advantage since it allowed me to stumble upon eM Client which doesn't induce those crashes and then turned out to be a more pleasant user experience than Apple's own native Mail app, which I would have otherwise never considered leaving in favor of an add-on.

Miscellany

"Build with Andrew"

I have two pieces I want to share next, then I'd like to cue Leo, since Leo recently had his own Claude Code-related revelation. But first:

One of the AI newsletters I keep an eye on is called "The Batch". It's published by DeepLearning.AI. Last Friday, an issue of "The Batch" arrived that caught my eye because I was pretty certain it would appeal to many of the non-coders who follow this podcast. That issue of the newsletter opened with:

> *Dear friends, We just launched a course that shows people who have never coded before, in less than 30 minutes, how to describe an idea for an app and build it using AI.*
>
> *It is now time for everyone — marketers, product professionals, operations specialists, analysts, students — to build software applications with AI! I've often spoken about why everyone should learn to code. I'm seeing a rapidly growing productivity gap between people who know how to code and those who don't. For many job roles I hire for, I now require at least basic coding knowledge. Many times, after I speak with a non-technical audience about the importance of building software using AI, people ask me how to get started. In the past, I didn't have a great answer. That motivated the DeepLearning.AI team to create "Build with Andrew." It's the best way for someone who wants to try vibe coding to get started!*
>
> *This course requires no prior knowledge of AI or coding. And it's vendor-agnostic. Specifically, learners can use these techniques with whatever tool they're most comfortable with (like ChatGPT, Gemini, Claude, or the chatbot built into the DeepLearning.AI platform).*

The "Andrew" cited here is Andrew Ng, the founder of DeepLearning.AI, the website that's hosting this free course. Andrew also co-founded Google Brain and Coursera and led AI at Baidu. He's an adjunct professor at Stanford University, formerly associate professor and Director of Stanford's AI Lab, SAIL. So, Andrew is certainly not some random YouTube influencer trying to get "Likes". To help everyone find this free 30-minute course, I've created a GRC shortcut using Andrew's first name, so: https://grc.sc/andrew. Just to be clear, I cannot vouch for this myself since I did not take the time to explore it. But Andrew is obviously the real deal, and it would certainly seem worthwhile for anyone who might have been wondering how to take the first steps toward AI-driven coding: https://www.deeplearning.ai/courses/build-with-andrew/

Al Liebl

> *Hi Steve, I've listened to the podcast for years and have thoroughly enjoyed it. I currently work in cloud security and find your content informative. Keep up the great work!*
>
> *I'm writing to you because of an open source project I've been working on. I should tell you, I am a terrible programmer. I'm 54, wear progressive lenses, hunt and peck and likely have mild ADD <g>. Having said that, I have been around computers since my dad built a Heath Kit H89 in our basement and I learned to use it. And as an adult I've worked for software and security companies in various roles so I know what good looks like and with my current role in cloud security I understand what gets attacked and how.*

I've grown tired of the lack of security, privacy and trust online and decided to start VettID. I've spent a bunch of time creating the design and then tried to figure out how to find people to help me get it built. That failed spectacularly <g>. So I dusted myself off and decided to go with plan B: AI.

A few months ago in my free time (I work full time) I went through some online classes for using AI for coding. They were helpful in teaching me the basics and I started using ChatGPT. It worked, kinda. I could prompt for what I wanted and ChatGPT would spit out the code and I would have to paste it into VSCode and do the things (build, commit, push, deploy, etc). In this situation I was still the clumsy bottle neck.

So I did a quick search and found Claude Code and it has been life changing! I pointed it at my repo and the rest is history. I trained Claude Code on my design and refined it. I had Claude Code develop a plan to implement it leveraging multiple Claude Code instances (one for iOS development, one for Android development, the main instance as the backend/frontend/lead and a Raspberry Pi as a tester). The plans were broken into issues assigned to each repo and the different instances could communicate via issues for troubleshooting. Working part time over the last few months I'm close to having the first version done. All the best, Al

This echoes what Leo has been saying and I think it goes to show that not all LLM AI are created equal. I've been chatting with ChatGPT about my code-related work, but Claude Code appears to have a significant edge there. But my needs are comparatively modest. I don't have a need for projects to be coded for me. I just want a quick reference source for code snippets here and there in languages with which I'm not fluent. And I've been extremely pleased with ChatGPT so far. So, Leo, I'd love to have you share your similar feelings about Claude Code

Listener Feedback

TJ Asher

Steve, I am all too familiar with the current state of the move to pure revenue generation by CAs around code signing. We were first bit by this with the change to HSM storage.

Our corporate policies prohibit allowing multiple users to access a computer under a common ID (which makes sense). Then, because one of our development environments installs some aspects "per user" under the HKEY_CURRENT_USER tree of the registry, our current licensing of certain add-ons would require an additional license for every possible user on that computer. So we're unable to implement an HSM to hold our code signing cert. As a result, we have no option but to store the key in the Cloud.

MS Azure has an option to allow this but I was informed by our IT group that this costs a minimum $30,000 to set up. Yes, that's the number we were told.

I'll interrupt TJ's note to mention that Rick Strahl's detailed "How to setup MS Azure code signing" blog posting might be someone he would like to show his IT group. They may have some other situation that imposes a $30,000 cost, but it's difficult to imagine what that might be. In any event, TJ's note continues:

The other option is for our CA to host it. But then we need to pay for each and every signature that happens. We have dozens and dozens of files that need to be signed frequently because we release updates every month. This quickly adds up, and you have to pre-pay for the signatures in blocks of 1000. That fee just went up and is now 26.4 cents per signature. $264.00 per 1000. And NO REFUNDS! Bought too many?, too bad, so sad. We go through enough signatures that we buy in blocks of 5000.

Let me just pause here to remind everyone just how absolutely and utterly insane this has become. Here's TJ's enterprise that needs to be performing lots of code signing. It should be a non-issue. They should be able to sign their code just as readily as they compile the code to be signed. But no. By making it increasingly difficult to sign code, for no good reason other than because they can, and by shortening certificate lifetimes, again, because the cabal of certificate authorities vote their own self-interests, the industry's certificate authorities are able to forcing everyone into a cloud-based service model where our use of our own signing key will be monitored and we will be charged a per-signing fee. TJ continues:

The CA group has the entire software industry over a barrel and there is naught we can do about it. Woe help you if you have a problem like we are experiencing now. Need good support? Buy a business account. It's no wonder this change to yearly expiration is happening, because they are going to lose out to free TLS certs from Let's Encrypt. They need to recoup that revenue somewhere. I can't imagine being a small or solo developer. Regards, TJ Asher

And then...

Jay Thompson

Are you interested in starting a service to issue certs? I put grccerts.com and grccodecerts.com on hold just in case. / jay thompson

First of all, Jay, I very much appreciate your consideration. There are many considerations. But addressing the name of such a service first, if I were to start a certificate authority I wouldn't tie it to GRC. It would need to have some sort of long-lived neutral name like DigiCert, IdenTrust or VeriSign. Those are good names for a certificate issuing authority.

I said "First of all", because there's more. There's a saying "Everyone has their own version of hell." In my case, having anything whatsoever to do with running or in any way managing a certificate authority would definitely be right up there near the top of the most hellacious ways for me to spend the remainder of my life. So, thanks but no thanks. I bitch and moan and carry on about the annoying cabal that has been allowed to form, but at the same time I deeply appreciate that there are people who are able to do what is completely beyond me. In the early days of GRC I ran a larger organization because I thought I was supposed to. And while my employees may have been happy, I was mostly miserable. Thanks to one rough Monday morning of firings, during which I reduced the company size by half, followed by a great deal of welcome attrition, I wound up with Sue to deal with operational stuff, Greg to run interference for me with tech support. That leaves me almost completely free to sit in a quiet corner by myself, with elevator music, a PC, mouse, screens and keyboard. That's my bliss.

But Jay's note brings up the interesting question of the contemporary creation of any new certificate authority. It's not a simple thing and it would require a concerted effort. But that said, I would imagine that the apparent greediness that is overtaking parts of the certificate authority business might be creating an opening for a well-financed newcomer.

The first problem any newcomer would encounter would be the establishment of their own root certificates into the heart of every single system where their signed certificates would need to be trusted. This might seem to be a classic chicken and egg problem since you cannot sell any certificate whose signature will not be trusted, and it'll be difficult to convince the various root store programs to accept any new and unproven root certificate without good cause.

Let's Encrypt solved this chicken and egg problem by "borrowing" the trust relationship IdenTrust had already long established. At Let's Encrypt's launch eleven years ago in 2015, its own root certificate was not present in client trust stores. So, in order to bootstrap trust, Let's Encrypt certificates were cross-signed by IdenTrust's well-trusted root certificate.

The way this worked was slick. Let's Encrypt first created its own new intermediate certificate. This intermediate certificate was used to sign all of Let's Encrypt's TLS web certificates. But the signatures of this intermediate certificate needed to be trusted by all of the world's web clients. To make that happen, Let's Encrypt's new intermediate certificate was co-signed – technically, it's called "cross signed" – by Let's Encrypts newly minted and not yet trusted root certificate, **and also** by IdenTrust's own already well-trusted root certificate.

The use of IdenTrust's root certificate to anchor the certificate chain meant that the signatures Let's Encrypt's intermediate certificate was placing on TLS certificates would be trusted from day one by **all** web clients, since those TLS web certificates "chained up" through Let's Encrypt's intermediate certificate to a root certificate everyone already trusted.

So, for the first three years, from 2015 to 2018, Let's Encrypt's certificate trust chain relied solely upon IdenTrust's root cross-signing. Then in 2018, I suppose only after Let's Encrypt had demonstrated the success of their concept, their own operational integrity, and the feasibility of their new ACME automation technology, their own root certificate which was named ISRG Root X1 was added to all of the major trust root stores. And finally, in 2021, three years after Let's Encrypt's root had been added to everyone's root stores, the root certificate that IdenTrust had used to cross-sign Let's Encrypt's intermediate certificate expired, ending the cross-sign phase and leaving all modern systems trusting Let's Encrypt's own root and the intermediate certificate it had signed.

The lesson taught by this bit of history is that creating a new trusted certificate-issuing authority is neither quick nor easy – not should it be. It would require an entity to demonstrate both their strict commitment to rule-following and their ability to rigorously follow those rules. They need to demonstrate that imbuing their signatures with widespread global trust would not in any way endanger the current status quo. If someone really wished to do so, they could arrange to bootstrap themselves into business the same way Let's Encrypt did. And I doubt that the members of the CA/Browser forum could prevent it from happening, much as they might not wish to have a powerful new lower-priced CA undercutting their well-established cash-printing business.

And taking the opposite viewpoint for a moment, we should all definitely require any upstart newcomer to prove themselves worthy of our trust. There's big money to be made in the certificate issuing business. The bigger the big guys get, the more dead weight overhead they accrue and need to carry, and the costlier their certificates become, the more tantalizing will be the opportunity for newcomers to attempt to get in for a piece of action. Anyone should have the ability to become a certificate authority in good standing, but as we've often noted, along with the right to print money comes the burden of being very careful whose certificates are signed and thus trusted.

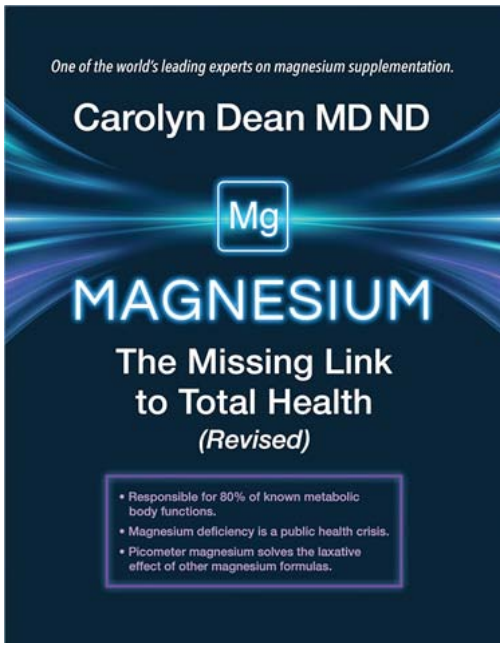
Scott

Steve, I've listened to you for years for your comments and sage advice about security matters and general comments about IT. Do want to say I appreciate your thoughts on vitamins and, after the latest podcast, have increased my intake of magnesium. Please continue to include the occasional thoughts about how vitamins might prolong our lives. Definitely not a bad diversion from the usual themes. Thanks. /Scott

I received similar sentiments from other listeners, and a younger listener said "Hey, supplementary nutrition is not just for older folks." So I will share what makes sense from time to time.

Steve Penfold

Hi Steve, Thank you for the info on magnesium last week. I've followed your previous leads on vitamin D3 (and now K2 as well), plus the ketogenic way of eating, but I wasn't aware of any of the magnesium 'stuff'.



Your book recommendation caused me to take a look at it on Amazon's site here in the UK. It seems that there have been two updates to 'The Magnesium Miracle' book by Carolyn Dean that you said you read in 2009. A quick summary of these updates:

- *In 2009, the version you read must have been the original 2003 version.*
- *There was an updated version released in 2017, with the same title.*
- *Additionally, there is now an even newer book, from 2023, billed as "an up-to-date summary that includes the advances in clinical magnesium research and therapy from the past 5 years." This is the version that I bought, in Kindle format, for only GBP £3.92 (equates to just over $5). Note that the word 'miracle' has been dropped from the title! - it's called "Magnesium: The Missing Link to Total Health (Revised)".*

Just though I'd let you know, in case you weren't aware that she's brought the 2003 information right up-to-date. I'll work through this book, so thanks again for the information, and the specific details on which formulations to take / avoid.

Thanks as always for all of the stuff that you and Leo do, and I'm more than happy for you to take the odd diversion into health, sci-fi, etc., from time to time on Security Now!

Steve Penfold - SpinRite owner - Club TWiT Member

Joey Albert

Thank you, Steve. You started me on The Lazarus project series yesterday and it is outstanding. Rotten Tomatoes meter is 100% fresh! Just bummed it's leaving Netflix this month. The 27th. I have to binge now. /Joey Albert

... and ...

Mr. Ron

Thanks for the tip about The Lazarus Project. I had never heard of it. I just finished attentively binging it, which really is the only way to follow the plot. It's the most outstanding time travel story I've ever seen.

Joey's and Mr. Ron's opinion was widely echoed among those who wrote. One listener reminded me of Apple TV's "For All Mankind" saying he thought it was fabulous. It's an interesting speculative fiction that extrapolates an alternative history where Russia wins the early stages of the space race by beating the US to the moon. Since I didn't watch the entire series I'm unable to offer my own opinion. Lorrie and I watched perhaps the first four or five episodes until we had caught up, but then we never returned for more. I don't recall it being amazing but I may not have given it a chance. It does rate an 8.1 on IMDB which is a very good score.

Philip

Hi Steve - Many thanks for the valuable low-down on reduction in the lifetime of code-signing certs. Does this mean that eventually all software will need to be updated every 2 years? What does that mean for software for which I've bought a perpetual licence, or freeware? What if it's no longer maintained? And I guess that you and I, probably the last two users of Paintshop Pro might have to find something else, at last! Best regards - Philip

Philip's quandary about this was echoed by a number of our listeners. Many who wrote in wondered how shorter code signing certificates would affect the long-term verifiable legitimacy of the code those certificates were used to sign. And right on cue, BleepingComputer posted a story last Wednesday with the headline: *"Logitech Options+, G HUB macOS apps break after certificate expires."* Whoops! BleepingComputer began their coverage of this, writing:

Logitech's Options+ and G Hub apps on macOS stopped working after their code-signing certificate expired, leaving users unable to launch them on Apple systems. Options+ is Logitech's input device configuration app, while G HUB is a similar app focused on customizing compatible Logitech G gaming peripherals. Both allow setting app profiles, button remapping, lighting options, scroll wheel, and sensor sensitivity.

Multiple users reported that Logitech apps on macOS did not load, making custom gestures, mappings, and scroll settings unavailable, and forcing them to use basic input functions. Impacted users expressed their frustration with the sudden loss of productivity-enhancing customizations, while many wasted time re-installing the Logitech apps, trying out Safe Mode, or wiping their configuration files.

Eventually, Logitech published a statement on its support portal, admitting that the issue was caused by a certificate that had expired.

At the same time, I signed GRC's Never10 windows executable program on Sunday, April 21st, 2019 and the code signing certificate I used to do that then expired on April 4th of 2022. Yet a check on the validity of Never10's code signing signature reveals that it remains valid to this day. So what's going on? This brings us to this week's topic, an examination of code signing certificate expiration and the answer to the question: How could Microsoft be issuing 3-day code signing certs?

3-Day Certificates

The title for today's podcast was inspired by a sentence that Rick Strahl casually dropped into his blog posting in passing. He noted: *"The certificates issued by Microsoft are very short lived - with expirations that last only 3 days to aggressively thwart invalid signing attacks in case a certificate is compromised."*

This raises the obvious question: How can it possibly be that Microsoft would be using code signing certificates that only last for three days before they expire?

The answer to that question brings us to a fundamental difference between the traditional web server authentication TLS certificates which we're all by now intimately familiar with, and code signing certificates which we've spent considerably less time exploring in the past. So what exactly are the differences between the two?

In the case of a web server's TLS certificate, our goal is to validate and authenticate the identity of a remote web server during a real-time transaction ***right now***. We need to be assured that the remote server we have just this moment connected to using its DNS-provided IP address is, in fact, the server we expect. DNS could have been compromised to lead us astray, or our Internet packet traffic could have been intercepted and diverted to a malicious web server.

To do this, we need to verify that the certificate we've just received over the connection we've just established matches the domain we intend to connect to and that the certificate is valid, not expired, in good standing, not revoked, and was signed by a certificate authority whose signatures we trust. If all of those things are true we would have every reason to believe that we're connecting to a web server for the domain we intend.

Okay. Now let's look at code signing.

The assurances we seek from signed code are obviously very different from the application of TLS web certificates. We want to ascertain two distinct things from the signature of any signed code: We want to determine the verifiable identity of the entity that signed the code, and we want to verify that not a single bit of the code that was signed has been altered since its signing. And that's it. That's the entire purpose of signed code: Who signed it and nothing has changed since.

We understand the general reason why certificates have expiration dates. While I complain quite a lot about certificate lifetimes being so short that their renewal becomes burdensome, at the same time it would be somewhat unnerving to be issued a trusted certificate that ***never*** expired. Yikes! If that certificate were to ever get loose at any time ***ever***, bad guys could abuse its trust, potentially forever. GRC has a code signing certificate stored, as they must all be now, in a SafeNet 5110 USB eToken. And it's actually sort of comforting to know that it comes with a drop-dead date after which it will become useless to anyone. If it didn't have that I would need to wipe its contents and then probably still smash it into tiny bits to make absolutely sure that it could never be reused.

But what about the code that it was used to sign? Let's take the Never10 for Windows executable I mentioned before. If you're curious you can go to GRC and download that executable right now to see for yourself. I signed that executable on Sunday, April 21st, 2019 using a code signing certificate that still had very nearly three years of life left on it, since it would expire on April 4th of 2022. At the time I signed that code, that certificate was in good standing. It was issued to my company, Gibson Research Corporation by DigiCert. The signing process meant that an unspoofable cryptographic hash was taken of the code, whereupon the private key I was in possession of was used to sign that hash, and GRC's certificate that was issued by DigiCert containing the matching public key was affixed to the end of the code.

From that moment on, anyone who obtained that Never10 code could check its certificate to see that the certificate was validly issued by DigiCert, a certificate authority that has carefully earned everyone's trust. The signature of the code's original hash could be verified using the public key contained in GRC's certificate, and that validly signed hash could be compared with a fresh hash of the code to verify that not a single bit of the original code had been changed after it was signed.

Remember the two assertions that are made through code signing: The identity of the certificate that performed the signing – in this case Gibson Research Corporation. And that since the time of the signing not a single bit has been changed.

Okay. Now jump forward to today, 2026. There's still a Never10 executable program that can be downloaded from GRC. And not a single bit of that code has been changed since the day it was signed in April of 2019. Yes, the certificate that was used to perform the signing expired three years later, which is almost four years ago, in April of 2022. But do we care? The signature accompanying that code remains valid. The certificate that's attached, still contains a public key that can be used to verify that not a single bit has changed since the moment it was originally signed, and Gibson Research Corporation's name is carried in the attached certificate.

Here's what's common between TLS and code signing certificates: In both cases the **only** requirement is that the certificate is valid at the time of its use.

So in the case of TLS that means that it must be valid and remain valid every time a web browser initiates a new connection and that certificate is offered up as proof of the remote server's identity.

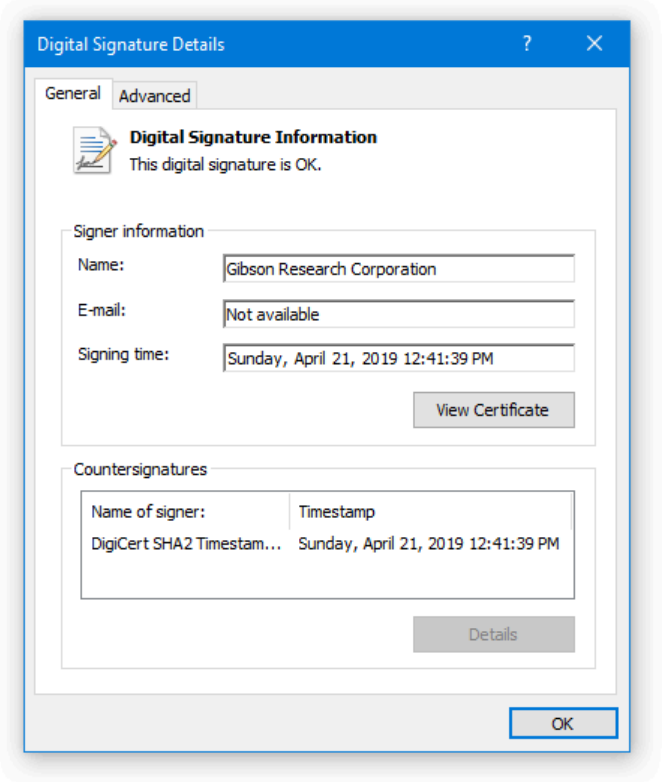
And in the case of Code Signing, the **only** requirement is that the certificate used to sign the code be valid **at the time** the code is signed. Since the only thing code signing is asserting is the identity of the signer and that nothing has changed since, requiring that the certificate be valid at the instant of the signing is sufficient. And now we can see why and how Microsoft's Azure code signing uses certificates having a very short 3-day life. Technically, it could be as short as a few hours. But creating certificates is not without some overhead. So I imagine they probably cache any certificates they've created for a couple of days in case the same signer returns for more signatures.

But there's an exploit we haven't yet addressed: What's to keep a bad guy who manages to get their mitts on someone else's expired certificate from using that certificate to sign their malicious

code? The signing certificate may have expired, but what's the enforcement mechanism for its expiration?

We might suggest that the PC used to perform the signing would examine the certificate and see that it had expired. Okay. The bad guys know that their stolen certificate has expired. So they simply turn back the clock on the signing PC they're using. Now the PC believes that the certificate is valid and in good standing. It has no way of knowing what day it is.

The obvious answer to this dilemma is for anyone who might be relying upon that signature to examine for themselves the signing certificate's expiration date & time – just as they would for a "real-time" TLS certificate – and refuse to trust anything signed by any certificate that has expired.



But now we have a new problem. As we've seen, what we ***really*** intend for code signing, is for any code that's signed by a certificate ***that is valid at the time of the signing*** to forever hence be judged as validly signed. So how do we accomplish that?

Introducing the TSA – the TimeStamp Authority.

A Timestamp Authority is a trusted 3rd-party. It's typically a certificate authority and is often, but not necessarily, the CA who provided the signing certificate. During the code signing process, once the code has been signed, the signature is hashed and forwarded to a timestamp authority where it's bundled with a UTC-format timestamp and that package is signed with the timestamp authority's private key. They then return this signature along with their own TSA certificate containing their public key. The result is a countersignature containing a verifiable timestamp.

The result of these machinations is that the final signed code contains two certificates: The code-signer's own certificate indicating their identity and the validity time of their certificate, and a signing timestamp that can be verified using the timestamp authority's certificate, which is also attached.

Now we have exactly what we want. The signing certificate's validity window, from not valid before to not valid after, is enforced by an unspoofable timestamp provided by a 3rd-party timestamping service whose certificate is also attached to allow their timestamp to be verified.

It's because GRC has always signed its code with the aid of a timestamping service that the validity of our apps never expire, even long after the certificate that was used to sign them is long gone.

So what happened with Logitech? The truth is, we don't know because we can't tell from what they've said.

They said that a certificate expired. But we don't definitely know what certificate expired. Adding a timestamp to executable code, libraries and such is now so routine that I'm skeptical that they could have somehow failed to do that. Timestamping everything ought to be deeply built into whatever signed their executable code.

I suspect that it's more likely that they have some sort of their own installer or patcher or updater or who-knows-what where they were using their own certificates internally in some fancy system of their own design and they tripped over their own tail. To me that seems more likely.

It's important to appreciate that it's only commercial certificate authorities who arbitrarily enforce short expiration policies. When you're creating your own certificates for your own internal purposes you can set whatever expiration date you like. So, someone may have created a 25-year certificate back in 2001, figuring that the system it was for would be replaced long before then. But we all know how that goes, right? So after a few years everyone completely forgot about it and never thought about it again, until – whoopsie!! – 25 years had flown past and that long-lived certificate surprised everyone by reaching its end-of-life date.

That seems like the most plausible explanation.

In any event, now everyone knows exactly what goes on with code signing certificates and how the static assertions they're designed to make differ from the real-time assertions made by TLS web certificates. And it should be clear how Microsoft's Azure cloud code signing service is able to sign with 3-day lifetime certificates. Those signatures are immediately timestamped while that short-lived code signing certificate is valid. After that, the certificate's expiration doesn't matter.